

EXERCICE 1

Représentation chaînée des listes circulaires

Une liste circulaire est une sorte de liste simplement (ou doublement) chaînée, qui comporte une caractéristique supplémentaire pour le déplacement dans la liste, "**elle n'a pas de fin**".

Pour rendre la liste sans fin, le pointeur **suivant** du dernier élément pointera sur le **1er élément** de la liste au lieu de la valeur NULL, que nous avons vu dans le cas des **listes non circulaires**.

Pour implémenter la liste circulaire, on utilise comme adresse :

- Soit l'adresse de la queue.
- Soit l'adresse de la tête et l'adresse de la queue.

Pour chaque implémentation :

- 1) Déterminer le nombre d'occurrence d'un élément donné dans une liste donnée.
- 2) Proposer un programme principal pour tester l'algorithme ci-dessus.

EXERCICE 2

Les Files

Une file est une structure de données de type FIFO (First In First Out) : le premier entré est le premier sorti. Les primitives de cette structure sont :

- initFile : initialise une file
- estVideF(f) : renvoie vrai si la file est, faux sinon
- estPleinF(f) : renvoie faux ; //on suppose que la file n'est jamais pleine.
- sommetFile (f) retourne l'élément au sommet de la file si la file est non vide (sans modifier l'état de la file).
- enfiler(elt, f) : ajout un élément en queue de la file si la file n'est pas pleine
- défiler(f) : supprime de la file le premier élément si la file est non vide.

Pour implémenter une file on utilise :

- Soit une liste simplement chaînée caractérisé uniquement avec l'adresse de la tête.
- Soit une liste simplement chaînée caractérisé avec deux adresses : l'adresse de la tête et l'adresse de la queue.
- Soit une liste circulaire caractérisé avec l'adresse de la queue.
- Soit par une liste circulaire caractérisé avec deux adresses : l'adresse de la tête et l'adresse de la queue.

- 1) Pour chaque structure, coder les différentes opérations qu'on applique sur une file.
- 2) Proposer un programme principal pour tester chacune des implémentations ci-dessus.

Les Piles

Une Pile est une structure de données de type LIFO (Last In First Out) : le dernier entré est le premier sorti. Les primitives de cette structure sont :

- initPile : initialise une Pile
- estVideP(p) : renvoie vrai si la pile est, faux sinon
- estPleinP(p) : renvoie faux ; //on suppose que la pile n'est jamais pleine.
- sommetPile (p) retourne l'élément au sommet de la pile si la pile est non vide (sans modifier l'état de la pile)
- empiler(elt, p) : ajout un élément en tête de la pile si la pile n'est pas pleine
- dépiler(p) : supprime de la pile le dernier élément si la pile est non vide.

Pour implémenter une Pile on utilise :

- Soit une représentation contigüe.
- Soit une liste simplement chaînée caractérisé uniquement avec l'adresse de la tête.
- Soit une liste simplement chaînée caractérisé avec deux adresses : l'adresse de la tête et l'adresse de la queue.
- Soit une liste circulaire caractérisé avec l'adresse de la tête.
- Soit une liste circulaire caractérisé avec deux adresses : l'adresse de la tête et l'adresse de la queue.

1) Pour chaque structure, coder les différentes opérations qu'on applique sur une pile.

2) Proposer un programme principal pour tester chacune des implémentations ci-dessus.

Application 1 File & Pile :

Ecrire un algorithme qui inverse une pile de réels. Doit-on utiliser une pile ou une file ?

Application 2 File & Pile :

Un palindrome est une chaîne de caractères qui se lit de la même manière de gauche à droite et de droite à gauche.

En utilisant les structures piles et files ainsi que leurs primitives écrire un algorithme qui détermine si une chaîne de caractères est un palindrome.

On suppose que la chaîne de caractères est passée en paramètre dans une liste simplement chaînée qui ne sera parcouru qu'une seule fois.

Ecrire un algorithme qui vérifie si une chaîne de caractères donnée est palindrome ou pas.